

Sistemas Distribuídos

(2026-2)

Capítulo 03: Processes

Introdução a *threads*

Idéia básica

Construir **processadores virtuais** em software, sobre processadores físicos:

Processador: Provê um conjunto de instruções, juntamente com a capacidade de executar, automaticamente, sequências dessas instruções.

Thread: Um processador mínimo implementado em software, em cujo **contexto** uma sequência de instruções pode ser executada. Salvar o contexto de uma *thread* implica em interromper a execução atual e salvar os dados necessários para continuar a execução posteriormente.

Processo: Um processador implementado em software em cujo contexto uma ou mais *threads* podem ser executadas. Executar uma *thread* significa que o processo executa a sequência de instruções associada dentro do contexto daquela *thread*.

Chaveamento (mudança) de contexto

Contextos

- **Contexto do processador:** Coleção mínima de valores armazenados nos registradores do processador, usados para execução de uma sequência de instruções (ex.: ponteiro de pilha, registradores de endereçamento, contador de programa).

Chaveamento (mudança) de contexto

Contextos

- **Contexto do processador**: Coleção mínima de valores armazenados nos registradores do processador, usados para execução de uma sequência de instruções (ex.: ponteiro de pilha, registradores de endereçamento, contador de programa).
- **Contexto de *thread***: Coleção mínima de valores armazenados em registradores e em memória, usados para a execução de uma sequência de instruções (i.e., contexto do processador + estado).

Chaveamento (mudança) de contexto

Contextos

- **Contexto do processador**: Coleção mínima de valores armazenados nos registradores do processador, usados para execução de uma sequência de instruções (ex.: ponteiro de pilha, registradores de endereçamento, contador de programa).
- **Contexto de *thread***: Coleção mínima de valores armazenados em registradores e em memória, usados para a execução de uma sequência de instruções (i.e., contexto do processador + estado).
- **Contexto de processo**: Coleção mínima de valores armazenados em registradores e em memória, usados para a execução de uma *thread* (i.e., contexto da *thread*, mas agora incluindo também, pelo menos, os valores dos registradores da MMU).

Chaveamento (mudança) de contexto

Observações

1. *Threads* compartilham o mesmo espaço de endereçamento. O chaveamento de contexto de *threads* pode ser feito de forma inteiramente independente do SO.
2. Chaveamento entre processos geralmente é mais dispendioso, pois envolve o SO (i.e., requer uma “*trap*” para o *kernel*).
3. Criar e destruir *threads* é bem mais barato do que criar e destruir processos.

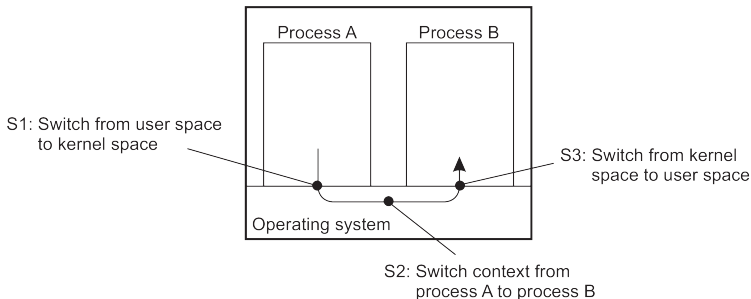
Por que usar threads

Algumas razões simples

- **Evitar bloqueios desnecessários:** um processo *single-thread* é **bloqueado** ao executar E/S; em um processo *multithread*, o sistema operacional pode alternar a CPU para outra *thread* do processo, sem bloqueá-lo.
- **Explorar o paralelismo:** as *threads* em um processo *multithread* podem ser programadas para serem executadas em paralelo em um multiprocessador ou processador *multicore*.
- **Evitar o chaveamento entre processos:** estruturar aplicações de grande porte não como uma coleção de processos, mas por meio de múltiplas *threads*.

Evitar chaveamento de processos

Evitar chaveamentos de contexto dispendiosos



Trade-offs (compensações)

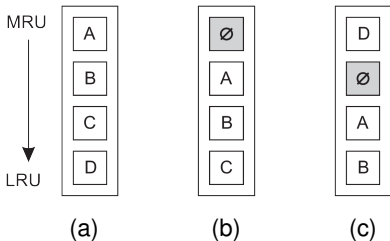
- *Threads* compartilham o mesmo espaço de endereçamento
- Não há suporte do SO/HW para proteger *threads* contra erros de acesso à memória umas das outras
- A troca de contexto de *threads* pode ser mais rápida do que a troca de contexto de processos

Os custos de chaveamento de contexto

Considere um tratador de interrupções de relógio simples

- **custos diretos**: o próprio chaveamento e a execução do tratador
- **custos indiretos**: outros custos, notadamente causados pela invalidação de caches

O que uma mudança de contexto pode causar: custos indiretos



- (a) antes da mudança de contexto
- (b) após a mudança de contexto
- (c) após acessar o bloco D.

Um exemplo simples em Python

```
1 from multiprocessing import Process
2 from time import *
3 from random import *
4
5 def sleeper(name):
6     t = gmtime()
7     s = randint(1,20)
8     txt = str(t.tm_min)+ ':' +str(t.tm_sec)+ ' '+name+ ' is going to sleep for '+str(s)+ ' seconds'
9     print(txt)
10    sleep(s)
11    t = gmtime()
12    txt = str(t.tm_min)+ ':' +str(t.tm_sec)+ ' '+name+ ' has woken up'
13    print(txt)
14
15 if __name__ == '__main__':
16     p = Process(target=sleeper, args=('eve',))
17     q = Process(target=sleeper, args=('bob',))
18     p.start(); q.start()
19     p.join(); q.join()
```

40:23 eve is going to sleep for 14 seconds

40:23 bob is going to sleep for 4 seconds

40:27 bob has woken up

40:37 eve has woken up

Um exemplo simples em Python

```

1  from multiprocessing import Process
2  from threading import Thread
3
4  shared_x = randint(10,99)
5
6  def sleeping(name):
7      global shared_x
8      t = gmtime(); s = randint(1,20)
9      txt = str(t.tm_min)+':'+str(t.tm_sec)+' '+name+' is going to sleep for '+str(s)+' seconds'
10     print(txt)
11     sleep(s)
12     t = gmtime(); shared_x = shared_x + 1
13     txt = str(t.tm_min)+':'+str(t.tm_sec)+' '+name+' has woken up, seeing shared x being '
14     print(txt+str(shared_x) )
15
16  def sleeper(name):
17     sleeplist = list()
18     print(name, 'sees shared x being', shared_x)
19     for i in range(3):
20         subsleeper = Thread(target=sleeping, args=(name+' '+str(i),))
21         sleeplist.append(subsleeper)
22
23     for s in sleeplist:
24         s.start()
25     for s in sleeplist:
26         s.join()
27     print(name, 'sees shared x being', shared_x)
28
29  if __name__ == '__main__':
30     p = Process(target=sleeper, args=('eve',))
31     g = Process(target=sleeper, args=('bob',))

```

Um exemplo simples em Python

```
eve sees shared x being 71
53:21 eve 0 is going to sleep for 20 seconds
bob sees shared x being 84
53:21 eve 1 is going to sleep for 15 seconds
53:21 eve 2 is going to sleep for 3 seconds
53:21 bob 0 is going to sleep for 8 seconds
53:21 bob 1 is going to sleep for 16 seconds
53:21 bob 2 is going to sleep for 8 seconds
53:24 eve 2 has woken up, seeing shared x being 72
53:29 bob 0 has woken up, seeing shared x being 85
53:29 bob 2 has woken up, seeing shared x being 86
53:36 eve 1 has woken up, seeing shared x being 73
53:37 bob 1 has woken up, seeing shared x being 87
bob sees shared x being 87
53:41 eve 0 has woken up, seeing shared x being 74
eve sees shared x being 74
```

Threads e sistemas operacionais

Questão principal

O SO deve prover suporte para *threads*? Ou *threads* devem ser implementados em pacotes no nível de usuário?

Solução no espaço do usuário

- Todas as operações podem ser tratadas completamente dentro de um único processo $\Rightarrow$ a implementação muito eficiente.
- Todos os serviços providos pelo *kernel* são em favor do processo no qual a *thread* reside $\Rightarrow$ se o *kernel* decidir bloquear uma *thread*, todo o processo será bloqueado.
- Threads são usadas quando há muitos eventos externos: *threads* bloqueiam para cada evento $\Rightarrow$ se o *kernel* não faz distinção entre *threads*, como ele pode ter suporte para sinalizar eventos para elas?

Threads e sistemas operacionais

Solução do kernel

A ideia principal é que o kernel contenha a implementação de um pacote de threads. Isso significa que todas as operações retornam como chamadas de sistema:

- Operações que bloqueiam uma thread não são mais um problema: o kernel agenda outra thread disponível dentro do mesmo processo.
- Lidar com eventos externos é simples: o kernel (que captura todos os eventos) agenda a thread associada ao evento.
- O problema é (ou costumava ser) a perda de eficiência, pois cada operação de thread requer uma interrupção para o kernel.

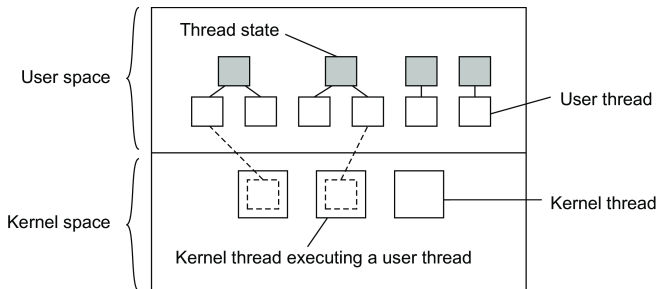
Conclusão – mas

Tentar misturar threads de nível de usuário e de nível de kernel em um único conceito; no entanto, o ganho de desempenho geralmente não compensa o aumento da complexidade.

Combinando threads de nível de usuário e de nível de kernel.

Ideia básica

Introduzir uma abordagem de threading de dois níveis: **threads do kernel** que podem executar threads de nível de usuário.



Threads de usuário e kernel combinadas

Operação principal

Threads de usuário e kernel combinadas

Operação principal

- A thread de usuário faz uma chamada de sistema $\Rightarrow$ a thread do kernel que está executando essa thread de usuário bloqueia. A thread de usuário permanece vinculada à thread do kernel.

Threads de usuário e kernel combinadas

Operação principal

- A thread de usuário faz uma chamada de sistema $\Rightarrow$ **a thread do kernel que está executando essa thread de usuário bloqueia**. A thread de usuário permanece **vinculada** à thread do kernel.
- O kernel pode **agendar outra thread do kernel que tenha uma thread de usuário executável vinculada a ela**. Observação: esta thread de usuário pode alternar para **qualquer** outra thread de usuário executável atualmente no espaço do usuário.

Threads de usuário e kernel combinadas

Operação principal

- A thread de usuário faz uma chamada de sistema $\Rightarrow$ **a thread do kernel que está executando essa thread de usuário bloqueia**. A thread de usuário permanece **vinculada** à thread do kernel.
- O kernel pode **agendar outra thread do kernel que tenha uma thread de usuário executável vinculada a ela**. Observação: esta thread de usuário pode alternar para **qualquer** outra thread de usuário executável atualmente no espaço do usuário.
- Uma thread de usuário chama uma operação de bloqueio em nível de usuário $\Rightarrow$ do context switch para uma thread de usuário executável (então vinculada à mesma thread do kernel).

Threads de usuário e kernel combinadas

Operação principal

- A thread de usuário faz uma chamada de sistema $\Rightarrow$ **a thread do kernel que está executando essa thread de usuário bloqueia**. A thread de usuário permanece **vinculada** à thread do kernel.
- O kernel pode **agendar outra thread do kernel que tenha uma thread de usuário executável vinculada a ela**. Observação: esta thread de usuário pode alternar para **qualquer** outra thread de usuário executável atualmente no espaço do usuário.
- Uma thread de usuário chama uma operação de bloqueio em nível de usuário $\Rightarrow$ do context switch para uma thread de usuário executável (então vinculada à mesma thread do kernel).
- Quando não há threads de usuário para agendar, uma thread do kernel pode permanecer ociosa e pode até mesmo ser removida (destruída) pelo kernel.

Utilizando threads no lado do cliente

Cliente web multithread

Ocultando latências de rede:

- O navegador web examina uma página HTML recebida e descobre que **mais arquivos precisam ser buscados**.
- **Cada arquivo é buscado por uma thread separada**, cada uma realizando uma requisição HTTP (bloqueante).
- À medida que os arquivos chegam, o navegador os exibe.

Múltiplas chamadas de requisição-resposta para outras máquinas (RPC)

- Um cliente realiza várias chamadas simultaneamente, cada uma por uma thread diferente.
- Em seguida, ele aguarda até que todos os resultados sejam retornados.
- Observação: se as chamadas forem para servidores diferentes, podemos ter um **aceleração linear**.

Cientes multithread: isso ajuda?

Paralelismo em nível de thread: TLP

Seja c_i a fração de tempo em que exatamente i threads estão sendo executadas simultaneamente.

$$TLP = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$$

com N sendo o número máximo de threads que (podem) ser executadas ao mesmo tempo.

Cientes multithread: isso ajuda?

Paralelismo em nível de thread: TLP

Seja c_i a fração de tempo em que exatamente i threads estão sendo executadas simultaneamente.

$$TLP = \frac{\sum_{i=1}^N i \cdot c_i}{1 - c_0}$$

com N sendo o número máximo de threads que (podem) ser executadas ao mesmo tempo.

Medidas práticas

Um navegador web típico tem um valor TLP entre 1,5 e 2,5 $\Rightarrow$ threads são usadas principalmente para **organizar logicamente** navegadores.

Usando threads no lado do servidor

Melhorar o desempenho

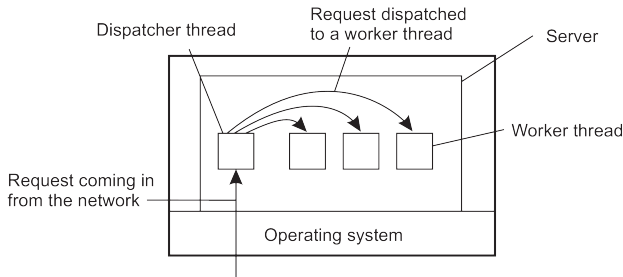
- Iniciar uma thread é mais barato do que iniciar um novo processo.
- Ter um servidor com uma única thread impede a fácil expansão para um sistema multiprocessador.
- Assim como nos clientes: **ocultar a latência da rede** reagindo à próxima solicitação enquanto a anterior está sendo respondida.

Melhor estrutura

- A maioria dos servidores tem altas demandas de E/S. Usar chamadas de bloqueio simples e **bem compreendidas** simplifica a estrutura.
- Programas multithread tendem a ser **menores e mais fáceis de entender** devido ao **fluxo de controle simplificado**.

Por que o multithreading é popular: organização

Modelo de despachante/trabalhador



Visão geral

Modelo	Características
Multithreading	Paralelismo, chamadas de sistema bloqueantes
Processo de thread única	Sem paralelismo, chamadas de sistema bloqueantes
Máquina de estados finitos	Paralelismo, chamadas de sistema não bloqueantes

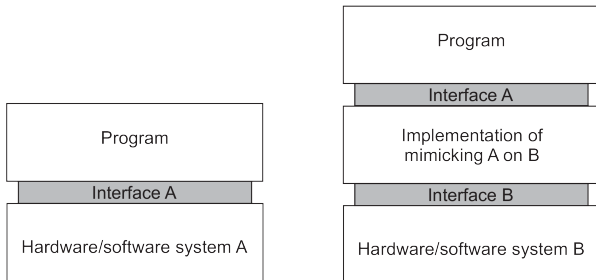
Virtualização

Observação

A virtualização é importante:

- Hardware **muda mais rápido** do que software
- Facilidade de **portabilidade** e migração de código
- **Isolamento** de componentes com falha ou atacados

Princípio: imitar interfaces

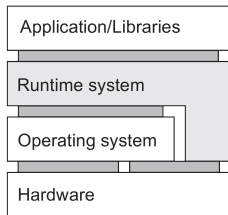


Imitando interfaces

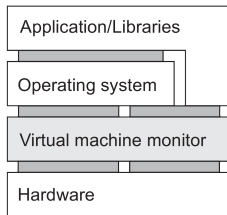
Quatro tipos de interfaces em três níveis diferentes

1. **Arquitetura do conjunto de instruções**: o conjunto de instruções de máquina, com dois subconjuntos:
 - Instruções privilegiadas: permitidas apenas para execução pelo sistema operacional.
 - Instruções gerais: podem ser executadas por qualquer programa.
2. **Chamadas de sistema** oferecidas por um sistema operacional.
3. **Chamadas de biblioteca**, conhecidas como **interface de programação de aplicativos** (API)

Formas de virtualização



(a) Máquina Virtual de Processo



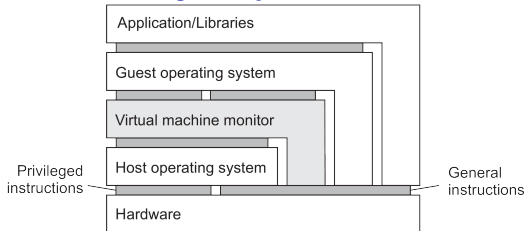
(b) Máquina Virtual de Modo de Exclusão Nativa

Diferenças

- (a) Conjunto separado de instruções, um interpretador/emulador, executado sobre um sistema operacional.
- (b) Instruções de baixo nível, juntamente com um sistema operacional mínimo e básico.
- (c) Instruções de baixo nível, mas delegando a maior parte do trabalho a um sistema operacional completo.

Aprofundando-se em VMs: desempenho

Refinando a organização



- **Instrução privilegiada:** se e somente se executada em modo usuário, causa uma **trap** para o sistema operacional
- **Instrução não privilegiada:** o restante

Instruções especiais

- **Instrução sensível ao controle:** pode afetar a configuração de uma máquina (por exemplo, uma que afeta o registrador de realocação ou a tabela de interrupções).
- **Instrução sensível ao comportamento:** o efeito é parcialmente determinado pelo contexto (por exemplo, `POPF` define um sinalizador habilitado para interrupção, mas apenas no modo de sistema).

Condição para virtualização

Condição necessária

Para qualquer computador convencional, um monitor de máquina virtual pode ser construído se o conjunto de instruções sensíveis para esse computador for um subconjunto do conjunto de instruções privilegiadas.

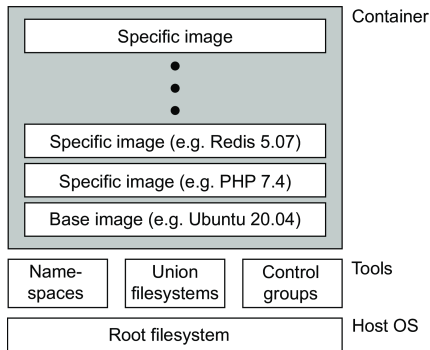
Problema: a condição nem sempre é satisfeita

Pode haver instruções sensíveis que são executadas no modo usuário sem causar uma interrupção no sistema operacional.

Soluções

- Emular todas as instruções
- Envolver instruções sensíveis não privilegiadas para desviar o controle para o VMM
- Paravirtualização: modificar o SO convidado, seja impedindo instruções sensíveis não privilegiadas ou tornando-as não sensíveis (ou seja, alterando o contexto).

Contêineres



- **Espaços de nomes:** uma coleção de processos em um contêiner recebe sua própria visão de identificadores.
- **Sistema de arquivos de união:** combine vários sistemas de arquivos em camadas, com apenas a camada mais alta permitindo operações de gravação (e aquela que faz parte de um contêiner).
- **Grupos de controle:** restrições de recursos podem ser impostas a uma coleção de processos.

Exemplo: PlanetLab

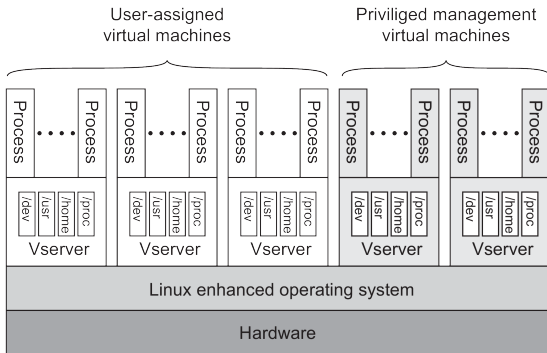
Essência

Diferentes organizações contribuem com máquinas, que posteriormente **compartilham** para diversos experimentos.

Problema

Precisamos garantir que diferentes aplicações distribuídas não interfiram umas nas outras. ⇒ **virtualização**

Organização básica do PlanetLab



Vserver

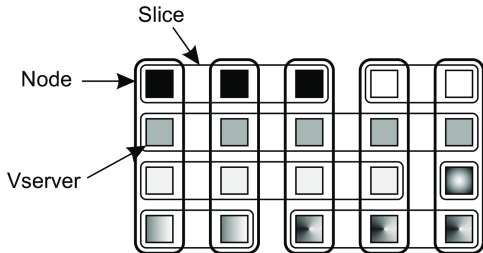
Ambiente independente e protegido com suas próprias bibliotecas, versões de servidor e assim por diante. Aplicações distribuídas são atribuídas a uma coleção de vservers distribuídos em várias máquinas

Servidores Virtuais e fatias do PlanetLab

Essência

- Cada servidor virtual opera em seu próprio ambiente (cf. `chroot`).
- Os aprimoramentos do Linux incluem o ajuste adequado dos IDs de processo (por exemplo, `init` com ID 0).
- Dois processos em servidores virtuais diferentes podem ter o mesmo ID de usuário, mas isso não implica que sejam o mesmo usuário.

A separação leva a fatias



VMs e computação em nuvem

Três tipos de serviços em nuvem

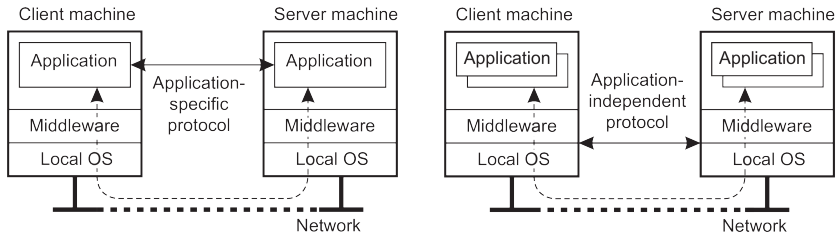
- **Infraestrutura como Serviço**, que abrange a infraestrutura básica
- **Plataforma como Serviço**, que abrange serviços de nível de sistema
- **Software como Serviço**, que contém os aplicativos reais

IaaS

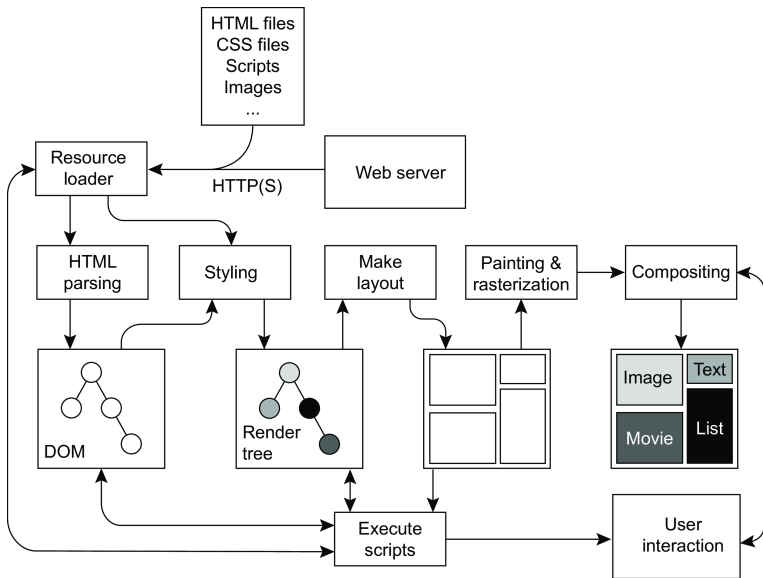
Em vez de alugar uma máquina física, um provedor de nuvem aluga uma VM (ou VMM) que pode estar compartilhando uma máquina física com outros clientes $\Rightarrow$ isolamento quase completo entre os clientes (embora o isolamento de desempenho possa não ser alcançado).

Interação client-servidor

Distinção entre protocolos de nível de aplicação e protocolos de middleware



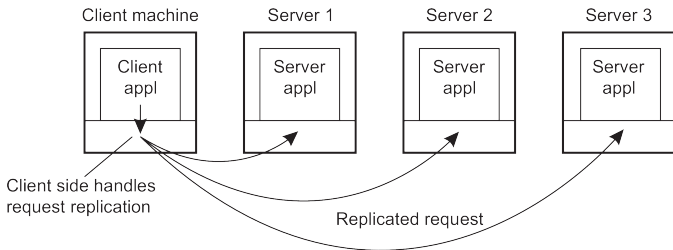
Exemplo de arquitetura de cliente: Web browser



Software do lado cliente

Geralmente o local para implementar transparência de distribuição

- **Transparência de acesso:** client-side stubs para RPCs
- **Transparência de localização/migração:** software/middleware no lado do cliente rastreia a localização
- **Transparência de replicação:** múltiplas chamadas tratadas pelo stub cliente:



- **Transparência de falha:** geralmente só pode ser implementada do lado do cliente (tenta mascarar falhas de servidor e de comunicação)

Servidores: Organização geral

Modelo básico

Um processo que implementa um serviço específico em nome de um conjunto de clientes. Ele aguarda uma requisição de um cliente e, em seguida, garante que a requisição seja atendida, após o que aguarda a próxima requisição (de algum cliente).

Servidores: Organização geral

Modelo básico

Um processo que implementa um serviço específico em nome de um conjunto de clientes. Ele aguarda uma requisição de um cliente e, em seguida, garante que a requisição seja atendida, após o que aguarda a próxima requisição (de algum cliente).

Dois tipos básicos

- **Servidor iterativo**: trata uma requisição antes de atender a próxima.
- **Servidor concorrente**: Usa um **dispatcher**, que pega a próxima requisição e a direciona para um processo/thread separado.

Observação

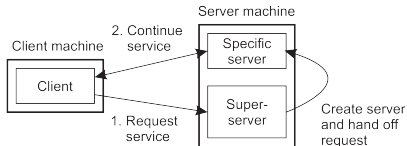
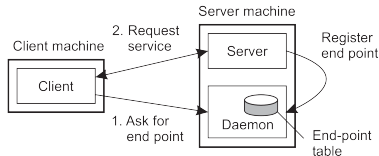
Servidores concorrentes são a norma podem lidar com múltiplas requisições simultaneamente, especialmente na presença de operações bloqueantes (acesso ao disco ou a outros servidores)

Como obter o endereço de contato de um servidor

Observação: serviços padrão são vinculados a uma porta específica

ftp-data	20	File Transfer [Default Data]
ftp	21	File Transfer [Control]
telnet	23	Telnet
smtp	25	Simple Mail Transfer
www	80	Web (HTTP)

Atribuição dinâmica de endpoints: duas abordagens



Comunicação fora da banda

Problema

É possível **interromper** um servidor uma vez que ele tenha aceitado (ou esteja no processo de aceitar) uma requisição de serviço?

Comunicação fora da banda

Problema

É possível interromper um servidor uma vez que ele tenha aceitado (ou esteja no processo de aceitar) uma requisição de serviço?

Solução 1: Usar uma porta separada para dados urgentes

- O servidor tem uma thread/processo separado para mensagens urgentes
- Ao receber uma mensagem urgente $\Rightarrow$ a requisição a ela associada é colocada em espera
- Obs.: necessita de suporte do SO para escalonamento com prioridades

Comunicação fora da banda

Problema

É possível interromper um servidor uma vez que ele tenha aceitado (ou esteja no processo de aceitar) uma requisição de serviço?

Solução 1: Usar uma porta separada para dados urgentes

- O servidor tem uma thread/processo separado para mensagens urgentes
- Ao receber uma mensagem urgente $\Rightarrow$ a requisição a ela associada é colocada em espera
- Obs.: necessita de suporte do SO para escalonamento com prioridades

Solução 2: Uso de um artifício da camada de transporte

- Exemplo: TCP permite mensagens urgentes na mesma conexão
- Mensagens urgentes podem ser recebidas por meio de interrupções (*signals*) do SO

Servidores e estado

Servidores sem estado

Nunca mantêm informação **precisa** sobre o status de um cliente após tratar uma requisição:

- Não mantêm um registro de arquivos abertos (simplesmente fecha o arquivo ao final do acesso)
- Não invalidam as caches dos clientes
- Não mantêm um registro de clientes

Servidores e estado

Servidores sem estado

Nunca mantêm informação **precisa** sobre o status de um cliente após tratar uma requisição:

- Não mantêm um registro de arquivos abertos (simplesmente fecha o arquivo ao final do acesso)
- Não invalidam as caches dos clientes
- Não mantêm um registro de clientes

Consequências

- Clientes e servidores são **completamente independentes**
- Reduz **inconsistências de estado** devidas a quebras (crashes) de cliente ou servidor
- Possível **perda de desempenho** porque, p. ex., um servidor não pode antecipar o comportamento dos clientes (ex.: não pode fazer pré-busca de blocos de arquivos)

Servidores e estado

Servidores sem estado

Nunca mantêm informação **precisa** sobre o status de um cliente após tratar uma requisição:

- Não mantêm um registro de arquivos abertos (simplesmente fecha o arquivo ao final do acesso)
- Não invalidam as caches dos clientes
- Não mantêm um registro de clientes

Consequências

- Clientes e servidores são **completamente independentes**
- Reduz **inconsistências de estado** devidas a quebras (crashes) de cliente ou servidor
- Possível **perda de desempenho** porque, p. ex., um servidor não pode antecipar o comportamento dos clientes (ex.: não pode fazer pré-busca de blocos de arquivos)

Pergunta relacionada com Redes 1

Comunicação orientada a conexões se encaixa no projeto de servidores sem

Servidores e estado

Servidores com estado (*stateful*)

Mantem um registro do status dos clientes:

- Arquivos abertos (possibilitando a pré-busca)
- Quais dados estão na cache dos clientes (permitindo que clientes mantenham cópias locais de dados compartilhados)

Servidores e estado

Servidores com estado (*stateful*)

Mantem um registro do status dos clientes:

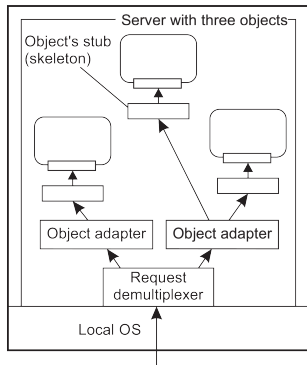
- Arquivos abertos (possibilitando a pré-busca)
- Quais dados estão na cache dos clientes (permitindo que clientes mantenham cópias locais de dados compartilhados)

Observação

O desempenho de servidores com estado pode ser extremamente alto, desde que os clientes possam manter cópias locais.

Confiabilidade: restauração do estado após falha do servidor.

Servidores de objetos



- **Política de ativação:** quais ações tomar ao chegar uma requisição:
 - Onde estão o código e os dados do objeto chamado?
 - Que modelo de threads usar?
 - Manter o estado modificado do objeto, se houver?
- **Adaptador de objetos:** implementa uma política de ativação específica

Exemplo: Ice runtime system – um servidor

```
1 import sys, Ice
2 import Demo
3
4 class PrinterI(Demo.Printer):
5     def __init__(self, t):
6         self.t = t
7
8     def printString(self, s, current=None):
9         print(self.t, s)
10
11 communicator = Ice.initialize(sys.argv)
12
13 adapter = communicator.createObjectAdapterWithEndpoints("SimpleAdapter", "default -p 11000")
14 object1 = PrinterI("Object1 says:")
15 object2 = PrinterI("Object2 says:")
16 adapter.add(object1, communicator.stringToIdentity("SimplePrinter1"))
17 adapter.add(object2, communicator.stringToIdentity("SimplePrinter2"))
18 adapter.activate()
19
20 communicator.waitForShutdown()
```

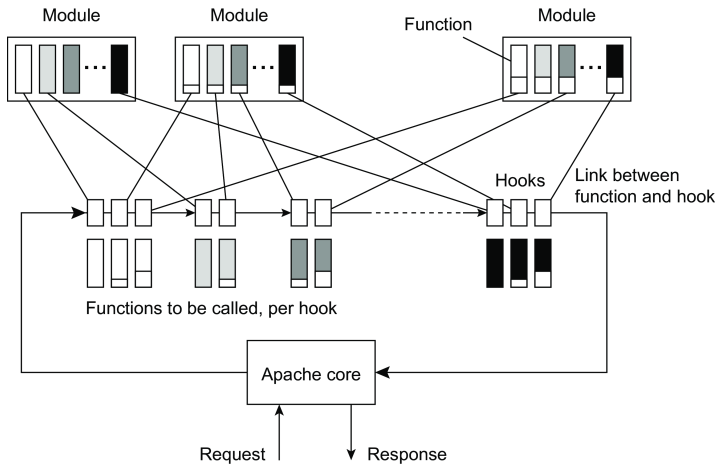
Example: Ice runtime system – a client

```
1 import sys, Ice
2 import Demo
3
4 communicator = Ice.initialize(sys.argv)
5
6 base1 = communicator.stringToProxy("SimplePrinter1:default -p 11000")
7 base2 = communicator.stringToProxy("SimplePrinter2:default -p 11000")
8 printer1 = Demo.PrinterPrx.checkedCast(base1)
9 printer2 = Demo.PrinterPrx.checkedCast(base2)
10 if (not printer1) or (not printer2):
11     raise RuntimeError("Invalid proxy")
12
13 printer1.printString("Hello World from printer1!")
14 printer2.printString("Hello World from printer2!")
15
16 communicator.waitForShutdown()
```

Object1 says: Hello World from printer1!

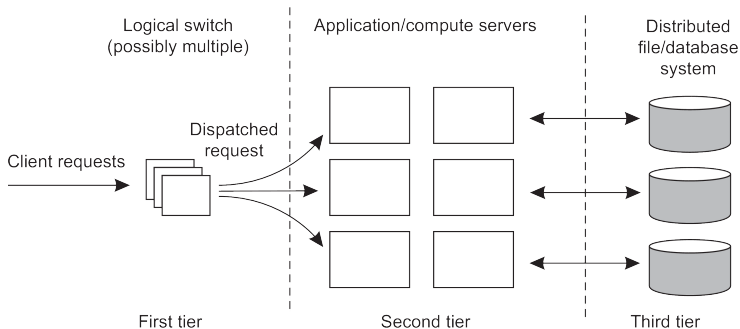
Object2 says: Hello World from printer2!

Example: the Apache Web server



Three different tiers

Common organization



Crucial element

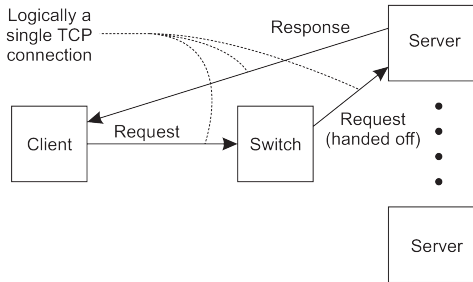
The first tier is generally responsible for passing requests to an appropriate server: **request dispatching**

Request Handling

Observation

Having the first tier handle all communication from/to the cluster may lead to a **bottleneck**.

A solution: TCP handoff



When servers are spread across the Internet

Observation

Spreading servers across the Internet may introduce administrative problems. These can be largely circumvented by using data centers from a single cloud provider.

Request dispatching: if locality is important

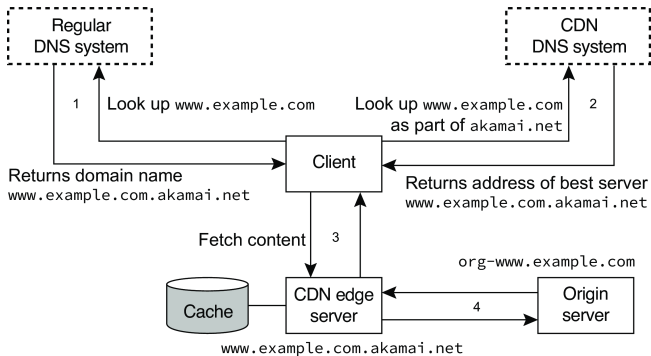
Common approach: use DNS:

1. Client looks up specific service through DNS - client's IP address is part of request
2. DNS server keeps track of replica servers for the requested service, and returns address of most local server.

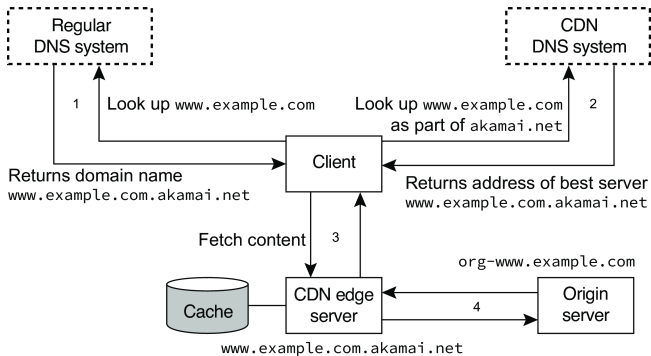
Client transparency

To keep client unaware of distribution, let DNS resolver act on behalf of client. Problem is that the resolver may actually be **far from local** to the actual client.

A simplified version of the Akamai CDN



A simplified version of the Akamai CDN



Important note

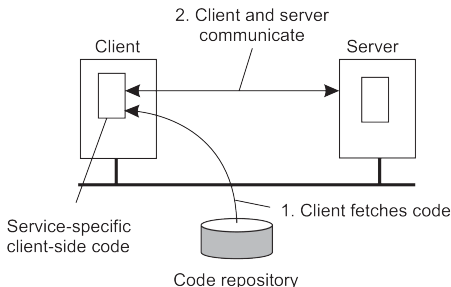
The cache is often sophisticated enough to hold more than just passive data. Much of the application code of the origin server can be moved to the cache as well.

Reasons to migrate code

Load distribution

- Ensuring that servers in a data center are **sufficiently** loaded (e.g., to prevent waste of energy)
- Minimizing communication by ensuring that computations are close to where the data is (think of mobile computing).

Flexibility: moving code to a client when needed



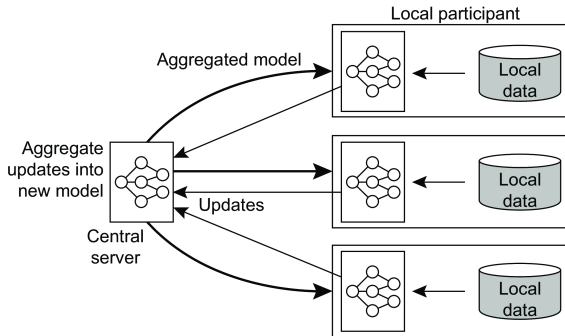
Avoids pre-installing software and increases dynamic configuration.

Reasons to migrate code

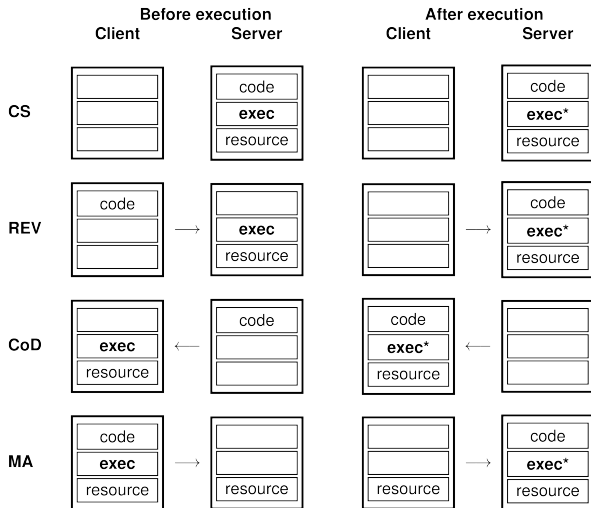
Privacy and security

In many cases, one cannot move data to another location, for whatever reason (often legal ones). **Solution:** move the code to the data.

Example: federated machine learning



Paradigms for code mobility



CS: Client-Server
CoD: Code-on-demand

REV: Remote evaluation
MA: Mobile agents

Strong and weak mobility

Object components

- **Code segment**: contains the actual code
- **Data segment**: contains the state
- **Execution state**: contains context of thread executing the object's code

Weak mobility: Move only code and data segment (and reboot execution)

- Relatively simple, especially if code is portable
- Distinguish **code shipping** (push) from **code fetching** (pull)

Strong mobility: Move component, including execution state

- **Migration**: move entire object from one machine to the other
- **Cloning**: start a clone, and set it in the same execution state.

Migration in heterogeneous systems

Main problem

- The target machine may not be **suitable to execute the migrated code**
- The definition of process/thread/processor context is **highly dependent on local hardware, operating system and runtime system**

Only solution: abstract machine implemented on different platforms

- Interpreted languages, effectively having their own VM
- Virtual machine monitors

Observation

As containers are directly dependent on the underlying operating system, their migration in heterogeneous environments is far from trivial, to simply impractical, just as process migration is.

Migrating a virtual machine

Migrating images: three alternatives

1. Pushing memory pages to the new machine and resending the ones that are later modified during the migration process.
2. Stopping the current virtual machine; migrate memory, and start the new virtual machine.
3. Letting the new virtual machine pull in new pages as needed: processes start on the new virtual machine immediately and copy memory pages on demand.

Performance of migrating virtual machines

Problem

A complete migration may actually take tens of seconds. We also need to realize that during the migration, a service will be completely unavailable for multiple seconds.

Measurements regarding response times during VM migration

